

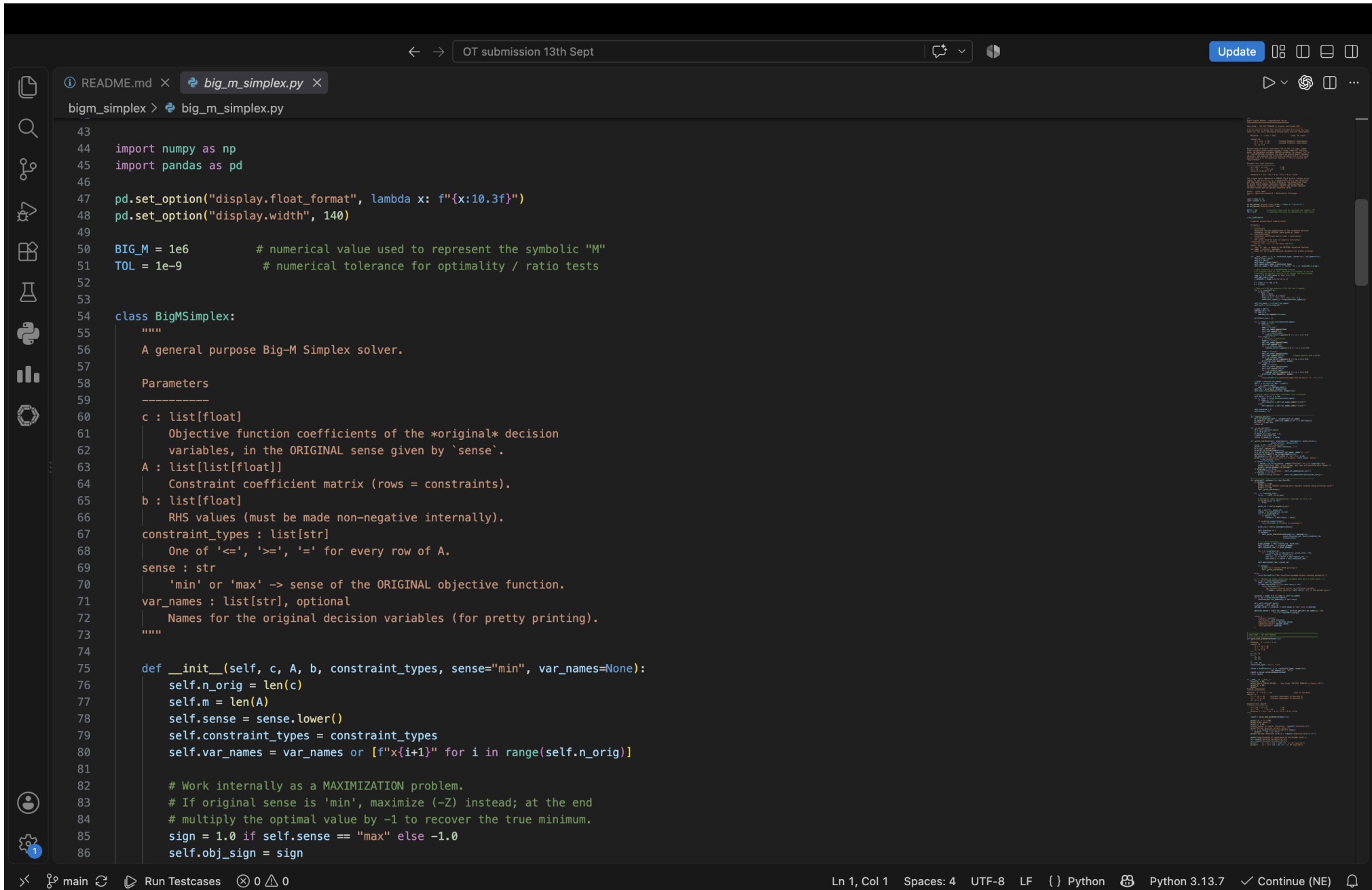
# Optimization Techniques — Assignment Submission

1. Big-M Simplex Method — The Diet Problem
2. Transportation Problem — VAM (Initial BFS) + MODI Method

*Code screenshots and corresponding program output*

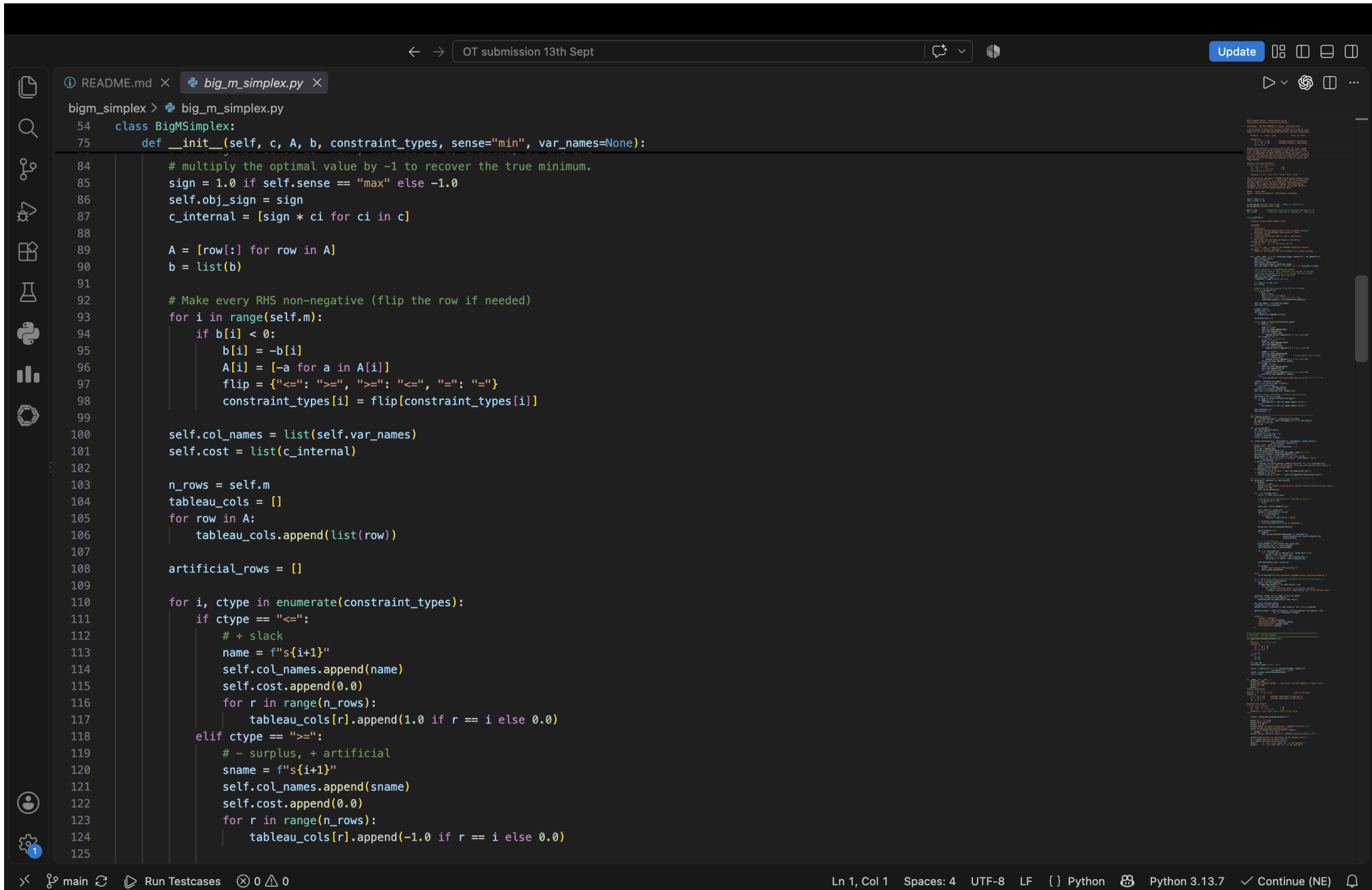
# **1. Big-M Simplex Method**

## Code (1/5): BigMSimplex class — docstring &amp; constructor setup



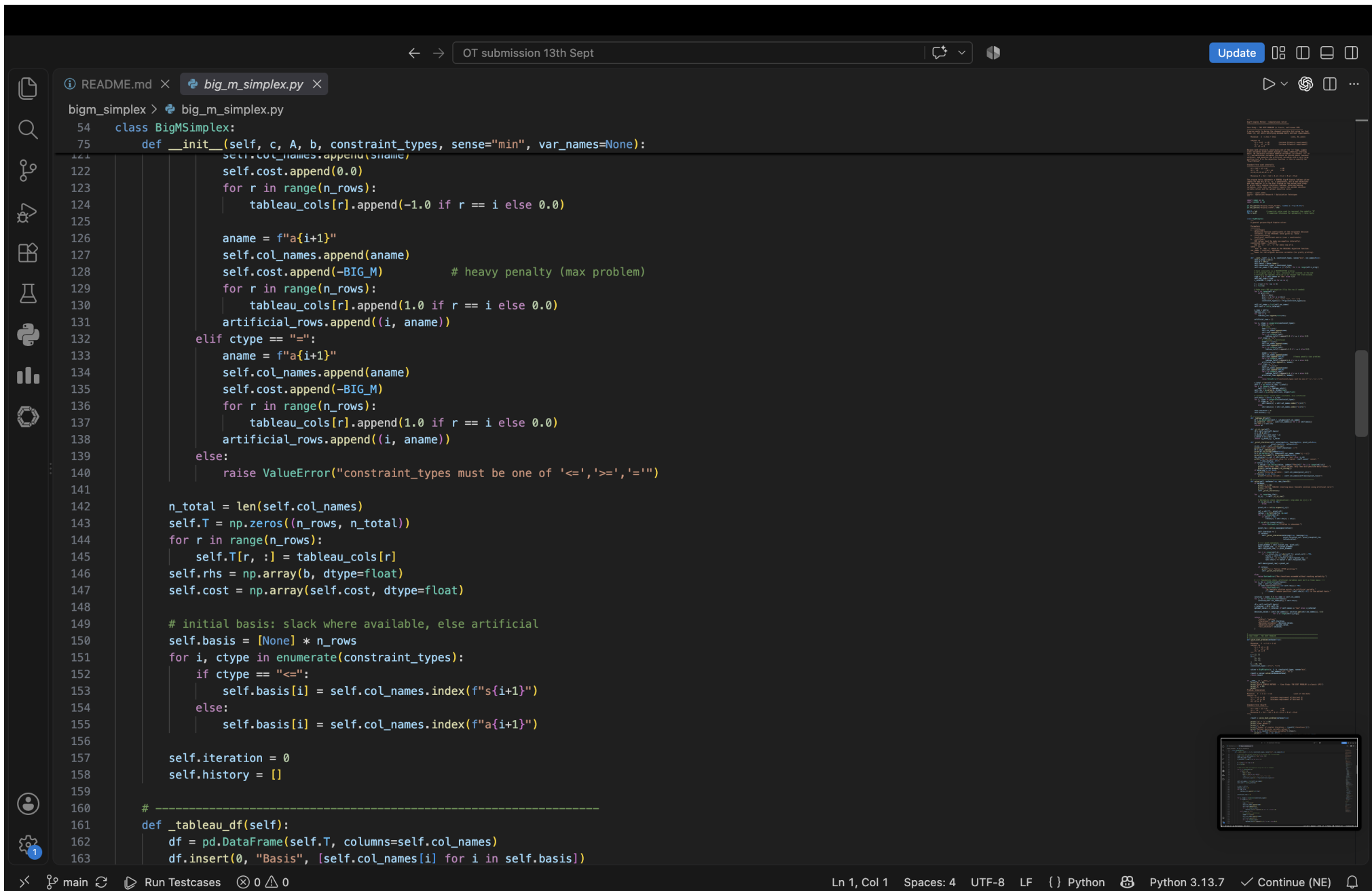
```
bigm_simplex > big_m_simplex.py
43
44 import numpy as np
45 import pandas as pd
46
47 pd.set_option("display.float_format", lambda x: f"{x:10.3f}")
48 pd.set_option("display.width", 140)
49
50 BIG_M = 1e6          # numerical value used to represent the symbolic "M"
51 TOL = 1e-9           # numerical tolerance for optimality / ratio tests
52
53
54 class BigMSimplex:
55     """
56     A general purpose Big-M Simplex solver.
57
58     Parameters
59     -----
60     c : list[float]
61         Objective function coefficients of the *original* decision
62         variables, in the ORIGINAL sense given by `sense`.
63     A : list[list[float]]
64         Constraint coefficient matrix (rows = constraints).
65     b : list[float]
66         RHS values (must be made non-negative internally).
67     constraint_types : list[str]
68         One of '<=', '>=', '=' for every row of A.
69     sense : str
70         'min' or 'max' -> sense of the ORIGINAL objective function.
71     var_names : list[str], optional
72         Names for the original decision variables (for pretty printing).
73     """
74
75     def __init__(self, c, A, b, constraint_types, sense="min", var_names=None):
76         self.n_orig = len(c)
77         self.m = len(A)
78         self.sense = sense.lower()
79         self.constraint_types = constraint_types
80         self.var_names = var_names or [f"x{i+1}" for i in range(self.n_orig)]
81
82         # Work internally as a MAXIMIZATION problem.
83         # If original sense is 'min', maximize (-Z) instead; at the end
84         # multiply the optimal value by -1 to recover the true minimum.
85         sign = 1.0 if self.sense == "max" else -1.0
86         self.obj_sign = sign
```

Code (2/5): building the standard-form tableau (RHS sign handling)



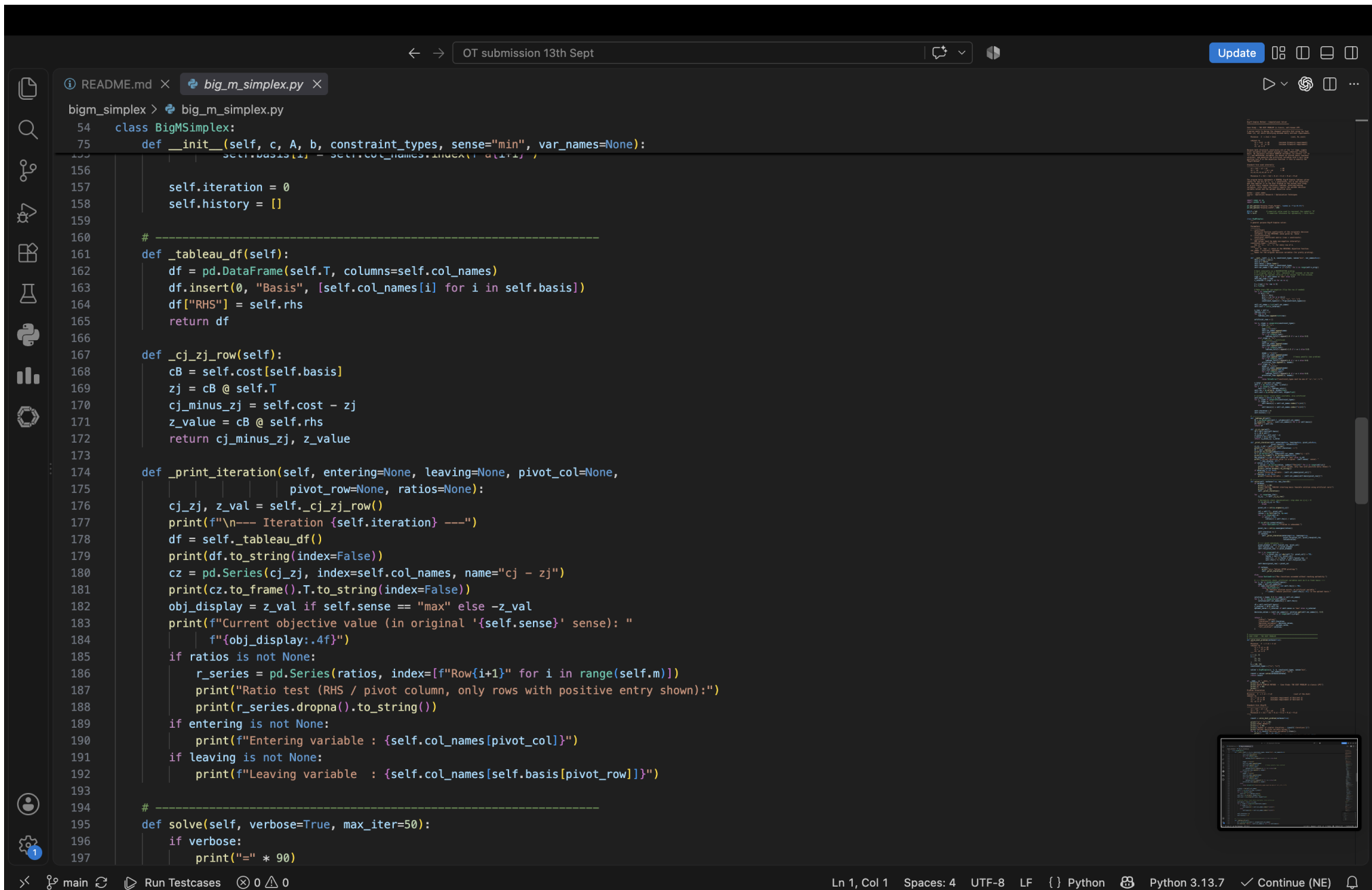
```
bigm_simplex > big_m_simplex.py
54 class BigMSimplex:
75     def __init__(self, c, A, b, constraint_types, sense="min", var_names=None):
84         # multiply the optimal value by -1 to recover the true minimum.
85         sign = 1.0 if self.sense == "max" else -1.0
86         self.obj_sign = sign
87         c_internal = [sign * ci for ci in c]
88
89         A = [row[:] for row in A]
90         b = list(b)
91
92         # Make every RHS non-negative (flip the row if needed)
93         for i in range(self.m):
94             if b[i] < 0:
95                 b[i] = -b[i]
96                 A[i] = [-a for a in A[i]]
97                 flip = {"<=": ">=", ">=": "<=", "=": "="}
98                 constraint_types[i] = flip[constraint_types[i]]
99
100         self.col_names = list(self.var_names)
101         self.cost = list(c_internal)
102
103         n_rows = self.m
104         tableau_cols = []
105         for row in A:
106             tableau_cols.append(list(row))
107
108         artificial_rows = []
109
110         for i, ctype in enumerate(constraint_types):
111             if ctype == "<=":
112                 # + slack
113                 name = f"s{i+1}"
114                 self.col_names.append(name)
115                 self.cost.append(0.0)
116                 for r in range(n_rows):
117                     tableau_cols[r].append(1.0 if r == i else 0.0)
118             elif ctype == ">=":
119                 # - surplus, + artificial
120                 sname = f"s{i+1}"
121                 self.col_names.append(sname)
122                 self.cost.append(0.0)
123                 for r in range(n_rows):
124                     tableau_cols[r].append(-1.0 if r == i else 0.0)
```

Code (3/5): adding slack / surplus / artificial variable columns



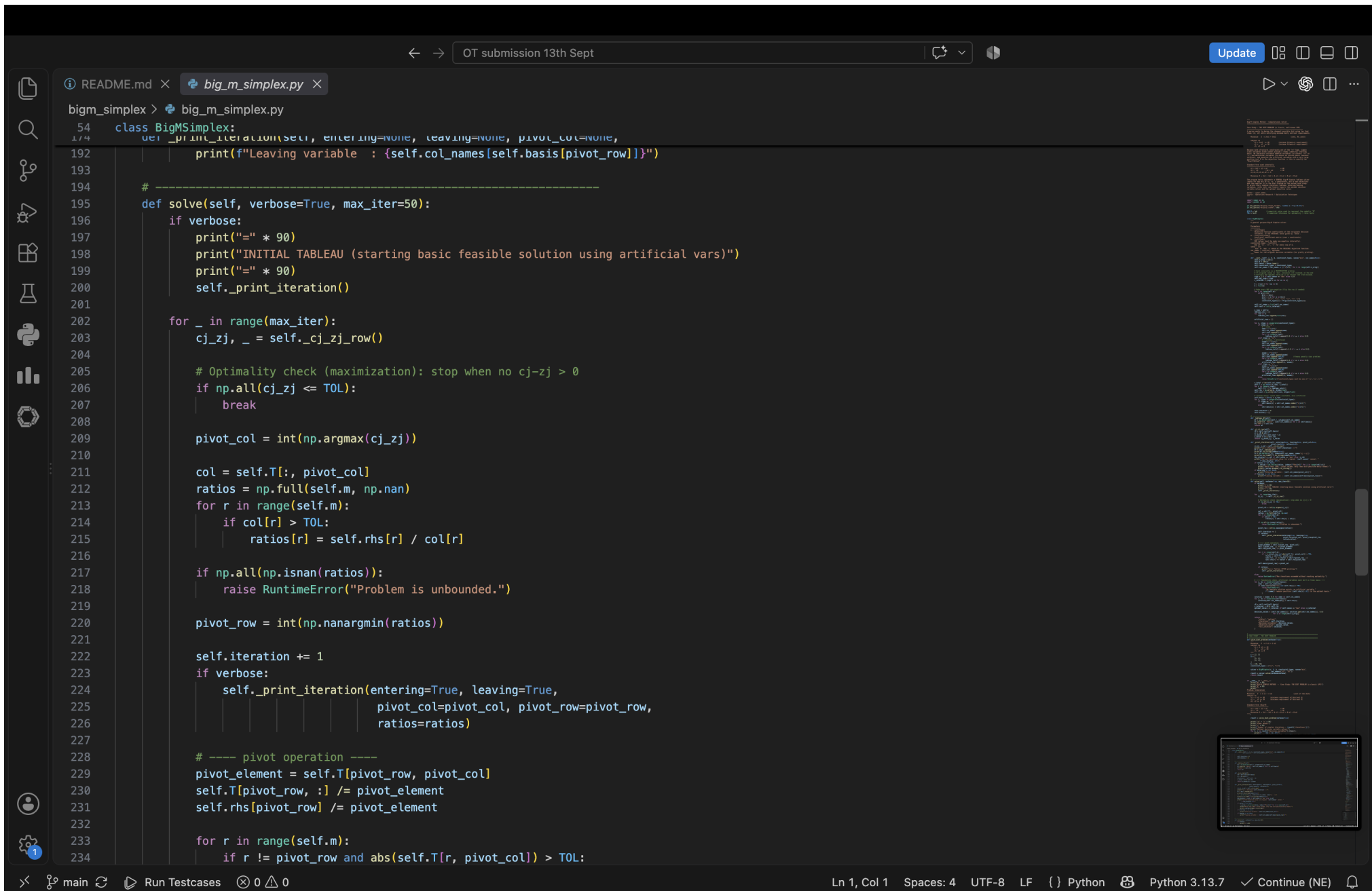
```
bigm_simplex > big_m_simplex.py
54 class BigMSimplex:
75     def __init__(self, c, A, b, constraint_types, sense="min", var_names=None):
121         self.col_names.append(sname)
122         self.cost.append(0.0)
123         for r in range(n_rows):
124             tableau_cols[r].append(-1.0 if r == i else 0.0)
125
126         aname = f"a{i+1}"
127         self.col_names.append(aname)
128         self.cost.append(-BIG_M) # heavy penalty (max problem)
129         for r in range(n_rows):
130             tableau_cols[r].append(1.0 if r == i else 0.0)
131         artificial_rows.append((i, aname))
132     elif ctype == "=":
133         aname = f"a{i+1}"
134         self.col_names.append(aname)
135         self.cost.append(-BIG_M)
136         for r in range(n_rows):
137             tableau_cols[r].append(1.0 if r == i else 0.0)
138         artificial_rows.append((i, aname))
139     else:
140         raise ValueError("constraint_types must be one of '<=', '>=', '='")
141
142     n_total = len(self.col_names)
143     self.T = np.zeros((n_rows, n_total))
144     for r in range(n_rows):
145         self.T[r, :] = tableau_cols[r]
146     self.rhs = np.array(b, dtype=float)
147     self.cost = np.array(self.cost, dtype=float)
148
149     # initial basis: slack where available, else artificial
150     self.basis = [None] * n_rows
151     for i, ctype in enumerate(constraint_types):
152         if ctype == "<=":
153             self.basis[i] = self.col_names.index(f"s{i+1}")
154         else:
155             self.basis[i] = self.col_names.index(f"a{i+1}")
156
157     self.iteration = 0
158     self.history = []
159
160     # -----
161     def _tableau_df(self):
162         df = pd.DataFrame(self.T, columns=self.col_names)
163         df.insert(0, "Basis", [self.col_names[i] for i in self.basis])
```

Code (4/5): Big-M penalty cost assignment &amp; initial basis



The screenshot displays a code editor with a dark theme. The top bar shows the file name 'OT submission 13th Sept' and an 'Update' button. The left sidebar contains icons for file explorer, search, and other IDE features. The main editor area shows the code for 'big\_m\_simplex.py'. The code defines a 'BigMSimplex' class with methods for initialization, tableau construction, row calculation, iteration printing, and solving the problem. The 'solve' method includes a verbose flag and a maximum iteration limit of 50. The bottom status bar indicates the current line and column (Ln 1, Col 1), the number of spaces (4), the encoding (UTF-8), the line feed style (LF), the file type (Python), the Python version (3.13.7), and the environment (Continue (NE)).

```
bigm_simplex > big_m_simplex.py
54 class BigMSimplex:
75     def __init__(self, c, A, b, constraint_types, sense="min", var_names=None):
156
157         self.iteration = 0
158         self.history = []
159
160     # -----
161     def _tableau_df(self):
162         df = pd.DataFrame(self.T, columns=self.col_names)
163         df.insert(0, "Basis", [self.col_names[i] for i in self.basis])
164         df["RHS"] = self.rhs
165         return df
166
167     def _cj_zj_row(self):
168         cB = self.cost[self.basis]
169         zj = cB @ self.T
170         cj_minus_zj = self.cost - zj
171         z_value = cB @ self.rhs
172         return cj_minus_zj, z_value
173
174     def _print_iteration(self, entering=None, leaving=None, pivot_col=None,
175                          pivot_row=None, ratios=None):
176         cj_zj, z_val = self._cj_zj_row()
177         print(f"\n--- Iteration {self.iteration} ---")
178         df = self._tableau_df()
179         print(df.to_string(index=False))
180         cz = pd.Series(cj_zj, index=self.col_names, name="cj - zj")
181         print(cz.to_frame().T.to_string(index=False))
182         obj_display = z_val if self.sense == "max" else -z_val
183         print(f"Current objective value (in original '{self.sense}' sense): "
184               f" {obj_display:.4f}")
185         if ratios is not None:
186             r_series = pd.Series(ratios, index=[f"Row{i+1}" for i in range(self.m)])
187             print("Ratio test (RHS / pivot column, only rows with positive entry shown):")
188             print(r_series.dropna().to_string())
189         if entering is not None:
190             print(f"Entering variable : {self.col_names[pivot_col]}")
191         if leaving is not None:
192             print(f"Leaving variable : {self.col_names[self.basis[pivot_row]]}")
193
194     # -----
195     def solve(self, verbose=True, max_iter=50):
196         if verbose:
197             print("=" * 90)
```

Code (5/5): tableau printing and  $c_j - z_j$  row computation

```
bigm_simplex > big_m_simplex.py
54 class BigMSimplex:
174     def _print_iteration(self, entering=None, leaving=None, pivot_col=None,
192         print(f"Leaving variable : {self.col_names[self.basis[pivot_row]]}")
193
194     # -----
195     def solve(self, verbose=True, max_iter=50):
196         if verbose:
197             print("=" * 90)
198             print("INITIAL TABLEAU (starting basic feasible solution using artificial vars)")
199             print("=" * 90)
200             self._print_iteration()
201
202         for _ in range(max_iter):
203             cj_zj, _ = self._cj_zj_row()
204
205             # Optimality check (maximization): stop when no cj-zj > 0
206             if np.all(cj_zj <= TOL):
207                 break
208
209             pivot_col = int(np.argmax(cj_zj))
210
211             col = self.T[:, pivot_col]
212             ratios = np.full(self.m, np.nan)
213             for r in range(self.m):
214                 if col[r] > TOL:
215                     ratios[r] = self.rhs[r] / col[r]
216
217             if np.all(np.isnan(ratios)):
218                 raise RuntimeError("Problem is unbounded.")
219
220             pivot_row = int(np.nanargmin(ratios))
221
222             self.iteration += 1
223             if verbose:
224                 self._print_iteration(entering=True, leaving=True,
225                                     pivot_col=pivot_col, pivot_row=pivot_row,
226                                     ratios=ratios)
227
228             # ---- pivot operation ----
229             pivot_element = self.T[pivot_row, pivot_col]
230             self.T[pivot_row, :] /= pivot_element
231             self.rhs[pivot_row] /= pivot_element
232
233             for r in range(self.m):
234                 if r != pivot_row and abs(self.T[r, pivot_col]) > TOL:
```

Ln 1, Col 1 Spaces: 4 UTF-8 LF {} Python Python 3.13.7 ✓ Continue (NE)

## Output (1/2): problem formulation, standard form &amp; initial tableau

```

OT submission 13th Sept
Update

bigm_simplex > my_output.txt
1 #####
2 BIG-M SIMPLEX METHOD - Case Study: THE DIET PROBLEM (a classic LPP)
3 #####
4
5 Problem formulation
6 =====
7 Minimize Z = 2 x1 + 3 x2 (cost of the diet)
8 subject to
9 x1 + 2 x2 >= 40 (minimum requirement of Nutrient A)
10 x1 + x2 >= 30 (minimum requirement of Nutrient B)
11 x1, x2 >= 0
12
13 Standard form (Big-M)
14 =====
15 x1 + 2x2 - s1 + a1 = 40
16 x1 + x2 - s2 + a2 = 30
17 Minimize Z = 2x1 + 3x2 + 0.s1 + 0.s2 + M.a1 + M.a2
18
19 =====
20 INITIAL TABLEAU (starting basic feasible solution using artificial vars)
21 =====
22
23 --- Iteration 0 ---
24 Basis x1 x2 s1 a1 s2 a2 RHS
25 a1 1.000 2.000 -1.000 1.000 0.000 0.000 40.000
26 a2 1.000 1.000 0.000 0.000 -1.000 1.000 30.000
27 | x1 x2 s1 a1 s2 a2
28 1999998.000 2999997.000 -1000000.000 0.000 -1000000.000 0.000
29 Current objective value (in original 'min' sense): 7000000.0000
30
31 --- Iteration 1 ---
32 Basis x1 x2 s1 a1 s2 a2 RHS
33 a1 1.000 2.000 -1.000 1.000 0.000 0.000 40.000
34 a2 1.000 1.000 0.000 0.000 -1.000 1.000 30.000
35 | x1 x2 s1 a1 s2 a2
36 1999998.000 2999997.000 -1000000.000 0.000 -1000000.000 0.000
37 Current objective value (in original 'min' sense): 7000000.0000
38 Ratio test (RHS / pivot column, only rows with positive entry shown):
39 Row1 20.000
40 Row2 30.000
41 Entering variable : x2
42 Leaving variable : a1
43
44 >>> Tableau AFTER pivoting:
45
46 --- Iteration 1 ---
47 Basis x1 x2 s1 a1 s2 a2 RHS
48 x2 0.500 1.000 -0.500 0.500 0.000 0.000 20.000
49 a2 0.500 0.000 0.500 -0.500 -1.000 1.000 10.000
50 | x1 x2 s1 a1 s2 a2
51 499999.500 0.000 499998.500 -1499998.500 -1000000.000 0.000
52 Current objective value (in original 'min' sense): 10000060.0000
53
54 --- Iteration 2 ---

```

Output (2/2): final iterations & optimal result ( $x_1=20$ ,  $x_2=10$ ,  $Z^*=70$ )

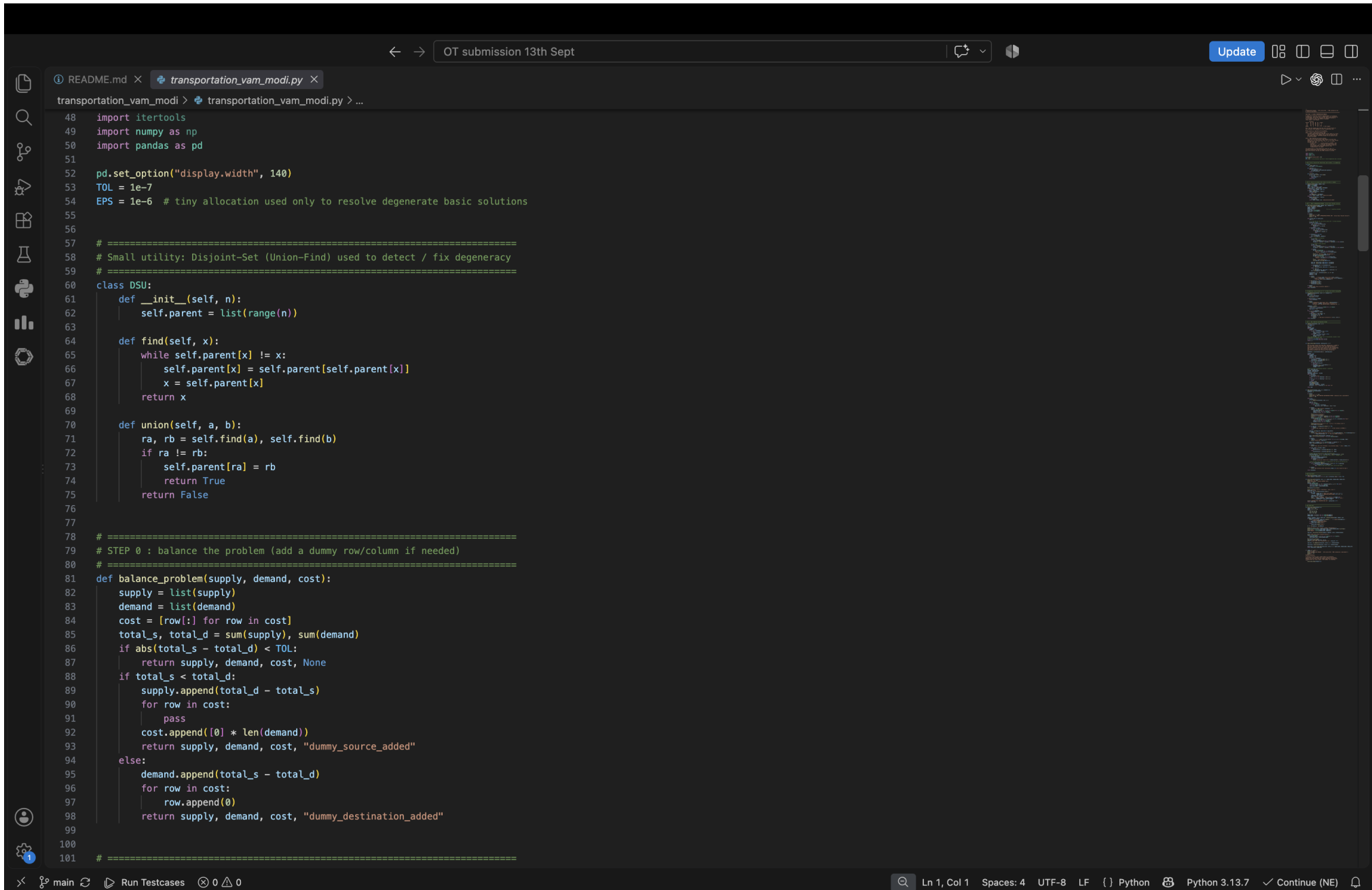
```

bigm_simplex > my_output.txt
36 1999998.000 2999997.000 -1000000.000 0.000 -1000000.000 0.000
37 Current objective value (in original 'min' sense): 7000000.0000
38 Ratio test (RHS / pivot column, only rows with positive entry shown):
39 Row1 20.000
40 Row2 30.000
41 Entering variable : x2
42 Leaving variable : a1
43
44 >>> Tableau AFTER pivoting:
45
46 --- Iteration 1 ---
47 Basis x1 x2 s1 a1 s2 a2 RHS
48 x2 0.500 1.000 -0.500 0.500 0.000 0.000 20.000
49 a2 0.500 0.000 0.500 -0.500 -1.000 1.000 10.000
50 x1 x2 s1 a1 s2 a2
51 499999.500 0.000 499998.500 -1499998.500 -1000000.000 0.000
52 Current objective value (in original 'min' sense): 10000060.0000
53
54 --- Iteration 2 ---
55 Basis x1 x2 s1 a1 s2 a2 RHS
56 x2 0.500 1.000 -0.500 0.500 0.000 0.000 20.000
57 a2 0.500 0.000 0.500 -0.500 -1.000 1.000 10.000
58 x1 x2 s1 a1 s2 a2
59 499999.500 0.000 499998.500 -1499998.500 -1000000.000 0.000
60 Current objective value (in original 'min' sense): 10000060.0000
61 Ratio test (RHS / pivot column, only rows with positive entry shown):
62 Row1 40.000
63 Row2 20.000
64 Entering variable : x1
65 Leaving variable : a2
66
67 >>> Tableau AFTER pivoting:
68
69 --- Iteration 2 ---
70 Basis x1 x2 s1 a1 s2 a2 RHS
71 x2 0.000 1.000 -1.000 1.000 1.000 -1.000 10.000
72 x1 1.000 0.000 1.000 -1.000 -2.000 2.000 20.000
73 x1 x2 s1 a1 s2 a2
74 0.000 0.000 -1.000 -999999.000 -1.000 -999999.000
75 Current objective value (in original 'min' sense): 70.0000
76
77 =====
78 FINAL RESULT
79 =====
80 Number of simplex iterations : 2
81 Optimal decision variable values:
82 x1 = 20.0000
83 x2 = 10.0000
84 Optimal objective value Z* = 70.0000
85
86 Verification of constraints at the optimal point:
87 x1 + 2x2 = 40.0000 (>= 40 required)
88 x1 + x2 = 30.0000 (>= 30 required)
89

```

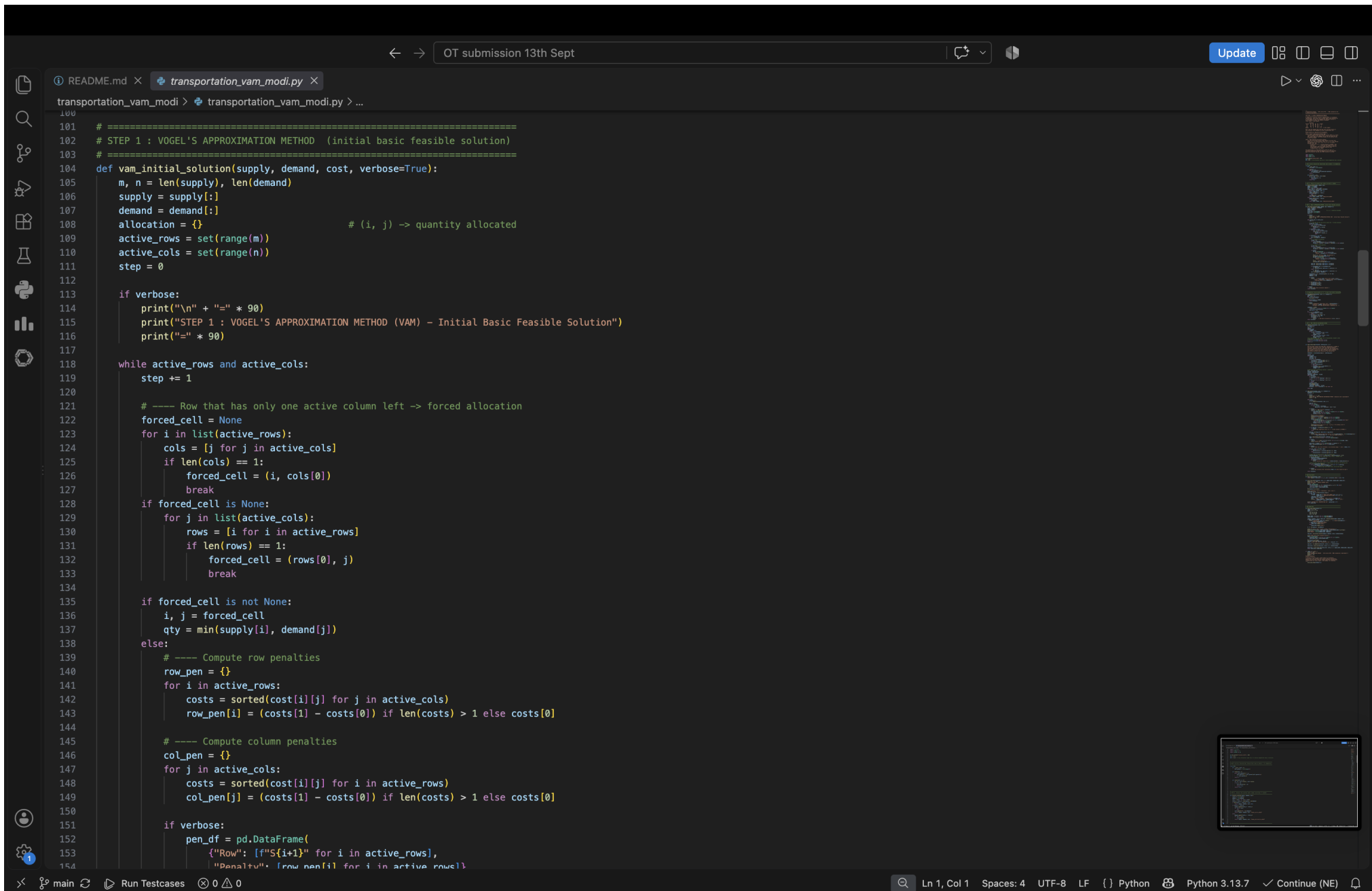
## **2. Transportation Problem — VAM + MODI**

Code (1/4): balancing the problem &amp; Disjoint-Set (Union-Find) utility



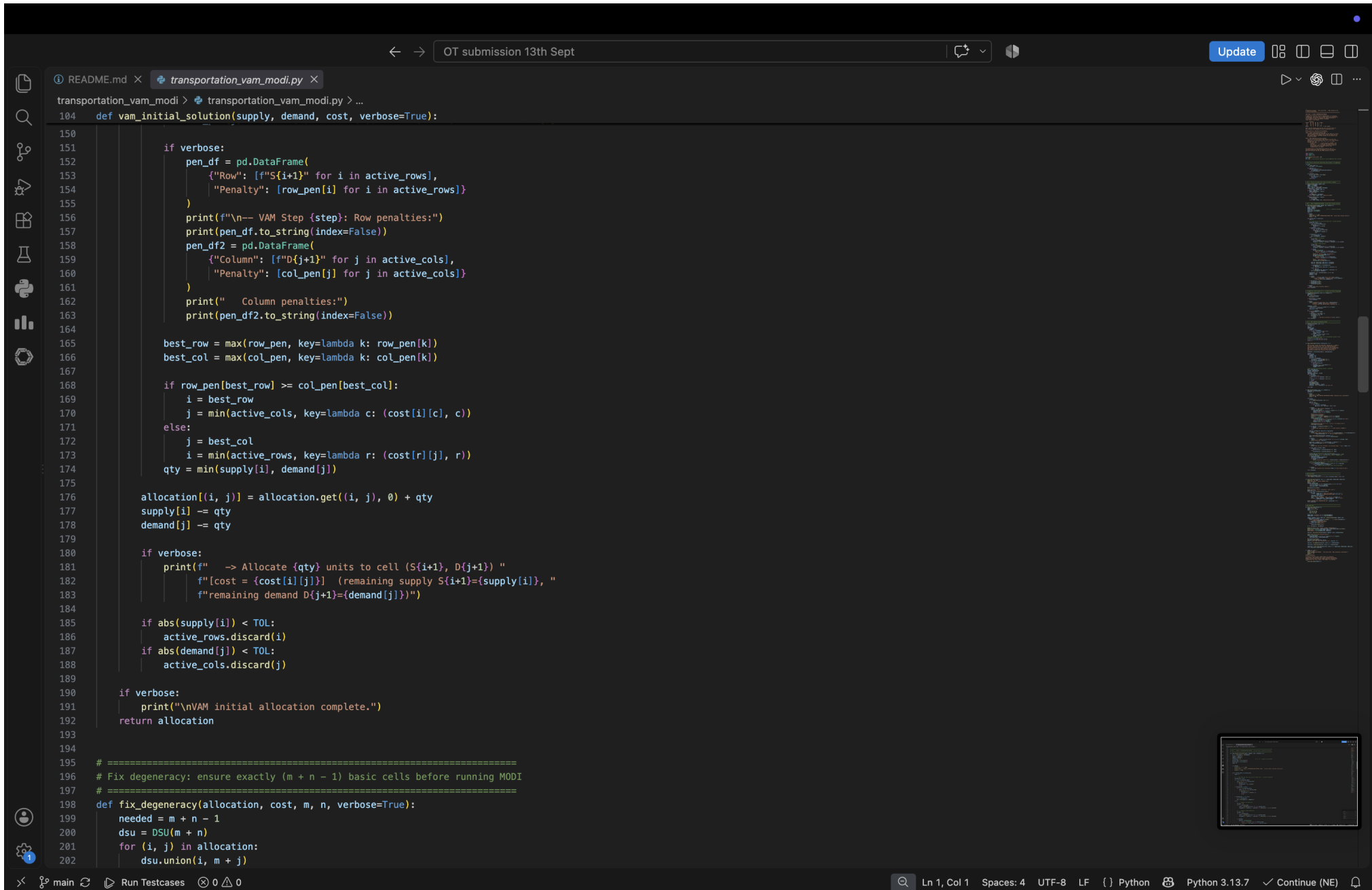
```
transportation_vam_modi.py
48 import itertools
49 import numpy as np
50 import pandas as pd
51
52 pd.set_option("display.width", 140)
53 TOL = 1e-7
54 EPS = 1e-6 # tiny allocation used only to resolve degenerate basic solutions
55
56 # =====
57 # Small utility: Disjoint-Set (Union-Find) used to detect / fix degeneracy
58 # =====
59 class DSU:
60     def __init__(self, n):
61         self.parent = list(range(n))
62
63     def find(self, x):
64         while self.parent[x] != x:
65             self.parent[x] = self.parent[self.parent[x]]
66             x = self.parent[x]
67         return x
68
69     def union(self, a, b):
70         ra, rb = self.find(a), self.find(b)
71         if ra != rb:
72             self.parent[ra] = rb
73             return True
74         return False
75
76 # =====
77 # STEP 0 : balance the problem (add a dummy row/column if needed)
78 # =====
79 def balance_problem(supply, demand, cost):
80     supply = list(supply)
81     demand = list(demand)
82     cost = [row[:] for row in cost]
83     total_s, total_d = sum(supply), sum(demand)
84     if abs(total_s - total_d) < TOL:
85         return supply, demand, cost, None
86     if total_s < total_d:
87         supply.append(total_d - total_s)
88         for row in cost:
89             pass
90         cost.append([0] * len(demand))
91         return supply, demand, cost, "dummy_source_added"
92     else:
93         demand.append(total_s - total_d)
94         for row in cost:
95             row.append(0)
96         return supply, demand, cost, "dummy_destination_added"
97
98 # =====
99
100
101 # =====
```

## Code (2/4): Vogel's Approximation Method — penalty computation



```
100
101 # =====
102 # STEP 1 : VOGEL'S APPROXIMATION METHOD (initial basic feasible solution)
103 # =====
104 def vam_initial_solution(supply, demand, cost, verbose=True):
105     m, n = len(supply), len(demand)
106     supply = supply[:]
107     demand = demand[:]
108     allocation = {} # (i, j) -> quantity allocated
109     active_rows = set(range(m))
110     active_cols = set(range(n))
111     step = 0
112
113     if verbose:
114         print("\n" + "=" * 90)
115         print("STEP 1 : VOGEL'S APPROXIMATION METHOD (VAM) - Initial Basic Feasible Solution")
116         print("=" * 90)
117
118     while active_rows and active_cols:
119         step += 1
120
121         # ---- Row that has only one active column left -> forced allocation
122         forced_cell = None
123         for i in list(active_rows):
124             cols = [j for j in active_cols]
125             if len(cols) == 1:
126                 forced_cell = (i, cols[0])
127                 break
128         if forced_cell is None:
129             for j in list(active_cols):
130                 rows = [i for i in active_rows]
131                 if len(rows) == 1:
132                     forced_cell = (rows[0], j)
133                     break
134
135         if forced_cell is not None:
136             i, j = forced_cell
137             qty = min(supply[i], demand[j])
138         else:
139             # ---- Compute row penalties
140             row_pen = {}
141             for i in active_rows:
142                 costs = sorted(cost[i][j] for j in active_cols)
143                 row_pen[i] = (costs[1] - costs[0]) if len(costs) > 1 else costs[0]
144
145             # ---- Compute column penalties
146             col_pen = {}
147             for j in active_cols:
148                 costs = sorted(cost[i][j] for i in active_rows)
149                 col_pen[j] = (costs[1] - costs[0]) if len(costs) > 1 else costs[0]
150
151             if verbose:
152                 pen_df = pd.DataFrame(
153                     {"Row": [f"S{i+1}" for i in active_rows],
154                      "Penalty": [row_pen[i] for i in active_rows]}
```

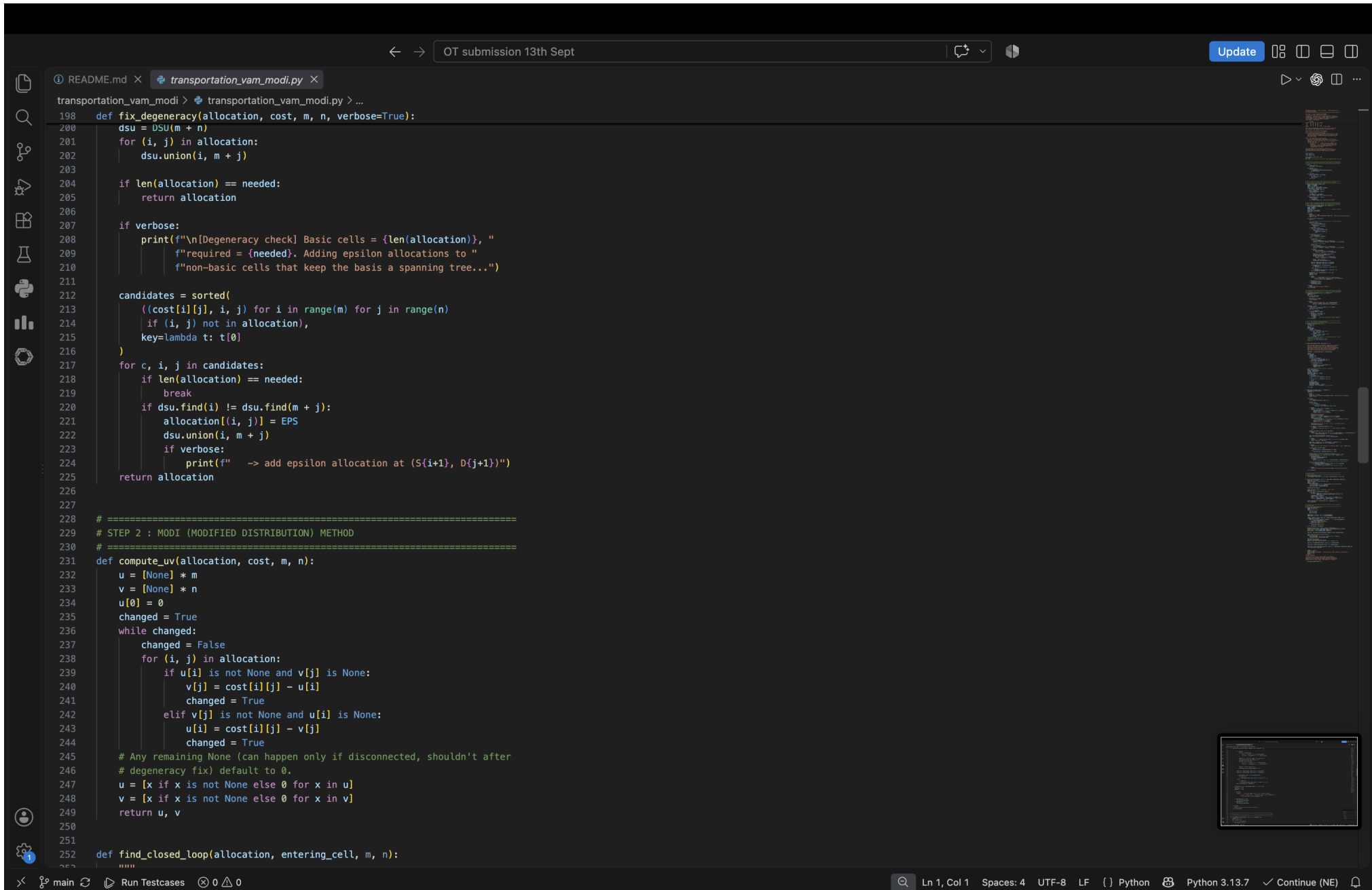
## Code (3/4): VAM allocation loop &amp; degeneracy-fix function



```

104 def vam_initial_solution(supply, demand, cost, verbose=True):
150
151     if verbose:
152         pen_df = pd.DataFrame(
153             {"Row": [f"S{i+1}" for i in active_rows],
154              "Penalty": [row_pen[i] for i in active_rows]}
155         )
156         print(f"\n-- VAM Step {step}: Row penalties:")
157         print(pen_df.to_string(index=False))
158         pen_df2 = pd.DataFrame(
159             {"Column": [f"D{j+1}" for j in active_cols],
160              "Penalty": [col_pen[j] for j in active_cols]}
161         )
162         print("    Column penalties:")
163         print(pen_df2.to_string(index=False))
164
165         best_row = max(row_pen, key=lambda k: row_pen[k])
166         best_col = max(col_pen, key=lambda k: col_pen[k])
167
168         if row_pen[best_row] >= col_pen[best_col]:
169             i = best_row
170             j = min(active_cols, key=lambda c: (cost[i][c], c))
171         else:
172             j = best_col
173             i = min(active_rows, key=lambda r: (cost[r][j], r))
174         qty = min(supply[i], demand[j])
175
176         allocation[(i, j)] = allocation.get((i, j), 0) + qty
177         supply[i] -= qty
178         demand[j] -= qty
179
180         if verbose:
181             print(f"    -> Allocate {qty} units to cell (S{i+1}, D{j+1}) "
182                   f"cost = {cost[i][j]} (remaining supply S{i+1}={supply[i]}, "
183                   f"remaining demand D{j+1}={demand[j]})")
184
185         if abs(supply[i]) < TOL:
186             active_rows.discard(i)
187         if abs(demand[j]) < TOL:
188             active_cols.discard(j)
189
190     if verbose:
191         print("\nVAM initial allocation complete.")
192     return allocation
193
194 # =====
195 # Fix degeneracy: ensure exactly (m + n - 1) basic cells before running MODI
196 # =====
197 def fix_degeneracy(allocation, cost, m, n, verbose=True):
198     needed = m + n - 1
199     dsu = DSU(m + n)
200     for (i, j) in allocation:
201         dsu.union(i, m + j)
202

```

Code (4/4): MODI method — computing  $u_i, v_j$  dual values

```
transportation_vam_modi.py
198 def fix_degeneracy(allocation, cost, m, n, verbose=True):
199     dsu = DSU(m + n)
200     for (i, j) in allocation:
201         dsu.union(i, m + j)
202
203     if len(allocation) == needed:
204         return allocation
205
206     if verbose:
207         print(f"\n[Degeneracy check] Basic cells = {len(allocation)}, "
208               f"required = {needed}. Adding epsilon allocations to "
209               f"non-basic cells that keep the basis a spanning tree...")
210
211     candidates = sorted(
212         ((cost[i][j], i, j) for i in range(m) for j in range(n)
213          if (i, j) not in allocation),
214         key=lambda t: t[0]
215     )
216
217     for c, i, j in candidates:
218         if len(allocation) == needed:
219             break
220         if dsu.find(i) != dsu.find(m + j):
221             allocation[(i, j)] = EPS
222             dsu.union(i, m + j)
223             if verbose:
224                 print(f"    -> add epsilon allocation at (S{i+1}, D{j+1})")
225     return allocation
226
227
228 # =====
229 # STEP 2 : MODI (MODIFIED DISTRIBUTION) METHOD
230 # =====
231 def compute_uv(allocation, cost, m, n):
232     u = [None] * m
233     v = [None] * n
234     u[0] = 0
235     changed = True
236     while changed:
237         changed = False
238         for (i, j) in allocation:
239             if u[i] is not None and v[j] is None:
240                 v[j] = cost[i][j] - u[i]
241                 changed = True
242             elif v[j] is not None and u[i] is None:
243                 u[i] = cost[i][j] - v[j]
244                 changed = True
245
246     # Any remaining None (can happen only if disconnected, shouldn't after
247     # degeneracy fix) default to 0.
248     u = [x if x is not None else 0 for x in u]
249     v = [x if x is not None else 0 for x in v]
250     return u, v
251
252 def find_closed_loop(allocation, entering_cell, m, n):
253     """
```

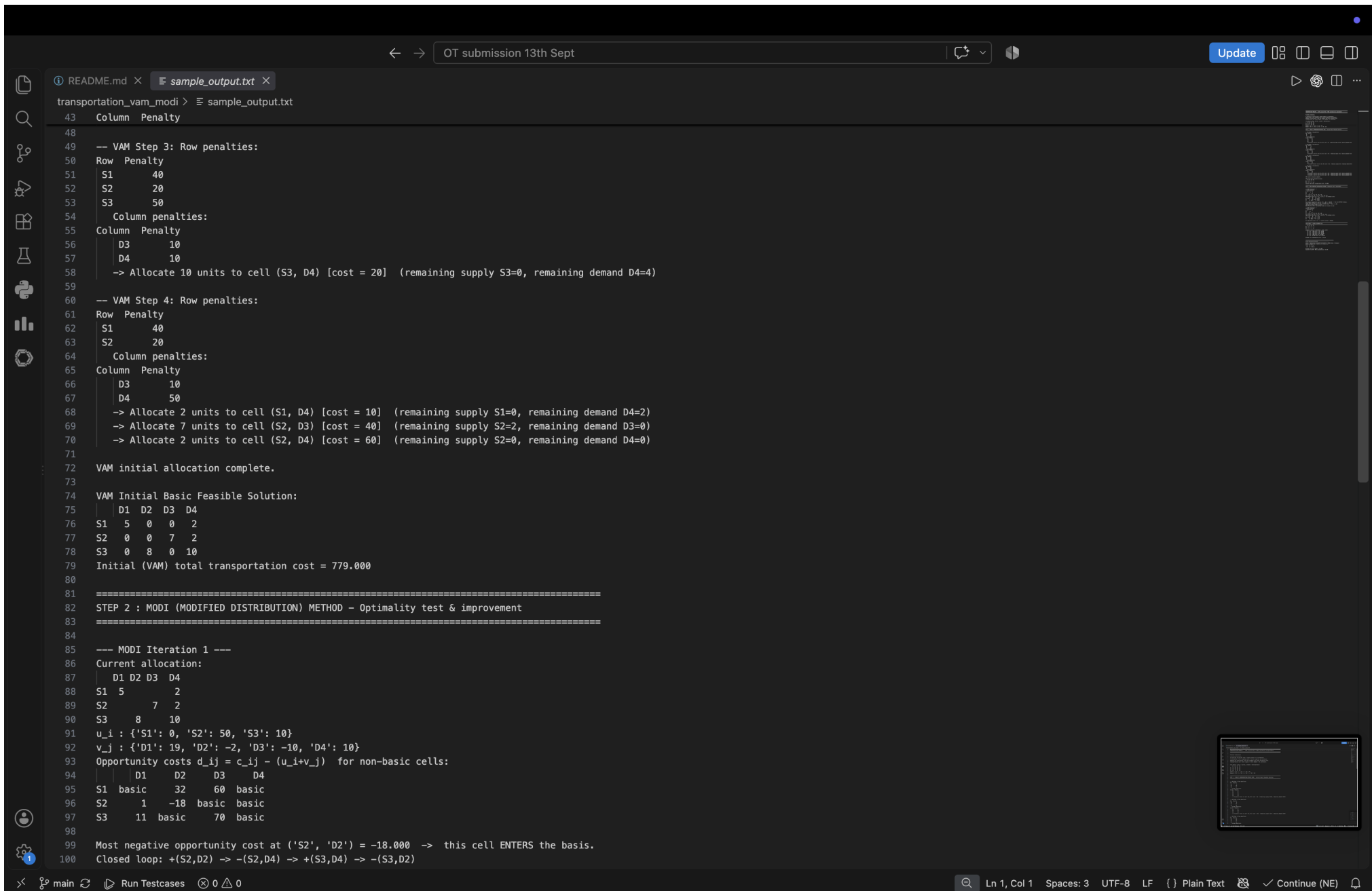
Output (1/4): problem data &amp; VAM Step 1–2 penalty tables

```

transportation_vam_modi > sample_output.txt
1 #####
2 TRANSPORTATION PROBLEM - VAM (initial BFS) + MODI (optimality & improvement)
3 #####
4
5 Problem formulation
6 =====
7 3 factories (S1,S2,S3) ship a single product to 4 warehouses
8 (D1,D2,D3,D4). Unit shipping costs, factory supplies and warehouse
9 demands are given below. Find the shipment plan that minimizes total
10 transportation cost (total supply = total demand = 34, balanced).
11
12 Cost matrix (rows = sources, columns = destinations):
13 | D1 D2 D3 D4
14 S1 19 30 50 10
15 S2 70 30 40 60
16 S3 40 8 70 20
17 Supply: {'S1': 7, 'S2': 9, 'S3': 18}
18 Demand: {'D1': 5, 'D2': 8, 'D3': 7, 'D4': 14}
19
20 =====
21 STEP 1 : VOGEL'S APPROXIMATION METHOD (VAM) - Initial Basic Feasible Solution
22 =====
23
24 -- VAM Step 1: Row penalties:
25 Row Penalty
26 S1 9
27 S2 10
28 S3 12
29 Column penalties:
30 Column Penalty
31 D1 21
32 D2 22
33 D3 10
34 D4 10
35 -> Allocate 8 units to cell (S3, D2) [cost = 8] (remaining supply S3=10, remaining demand D2=0)
36
37 -- VAM Step 2: Row penalties:
38 Row Penalty
39 S1 9
40 S2 20
41 S3 20
42 Column penalties:
43 Column Penalty
44 D1 21
45 D3 10
46 D4 10
47 -> Allocate 5 units to cell (S1, D1) [cost = 19] (remaining supply S1=2, remaining demand D1=0)
48
49 -- VAM Step 3: Row penalties:
50 Row Penalty
51 S1 40
52 S2 20
53 S3 50
54 Column penalties:

```

Output (2/4): VAM completes — initial BFS, cost = 779; MODI Iteration 1 begins



```

43 Column Penalty
44
45 -- VAM Step 3: Row penalties:
46 Row Penalty
47 S1 40
48 S2 20
49 S3 50
50 Column penalties:
51 Column Penalty
52 D3 10
53 D4 10
54 -> Allocate 10 units to cell (S3, D4) [cost = 20] (remaining supply S3=0, remaining demand D4=4)
55
56 -- VAM Step 4: Row penalties:
57 Row Penalty
58 S1 40
59 S2 20
60 Column penalties:
61 Column Penalty
62 D3 10
63 D4 50
64 -> Allocate 2 units to cell (S1, D4) [cost = 10] (remaining supply S1=0, remaining demand D4=2)
65 -> Allocate 7 units to cell (S2, D3) [cost = 40] (remaining supply S2=2, remaining demand D3=0)
66 -> Allocate 2 units to cell (S2, D4) [cost = 60] (remaining supply S2=0, remaining demand D4=0)
67
68 VAM initial allocation complete.
69
70 VAM Initial Basic Feasible Solution:
71 | D1 D2 D3 D4
72 S1 5 0 0 2
73 S2 0 0 7 2
74 S3 0 8 0 10
75 Initial (VAM) total transportation cost = 779.000
76
77 =====
78 STEP 2 : MODI (MODIFIED DISTRIBUTION) METHOD - Optimality test & improvement
79 =====
80
81 --- MODI Iteration 1 ---
82 Current allocation:
83 | D1 D2 D3 D4
84 S1 5 0 0 2
85 S2 0 0 7 2
86 S3 0 8 0 10
87 u_i : {'S1': 0, 'S2': 50, 'S3': 10}
88 v_j : {'D1': 19, 'D2': -2, 'D3': -10, 'D4': 10}
89 Opportunity costs d_ij = c_ij - (u_i+v_j) for non-basic cells:
90 | | D1 D2 D3 D4
91 S1 basic 32 60 basic
92 S2 1 -18 basic basic
93 S3 11 basic 70 basic
94
95 Most negative opportunity cost at ('S2', 'D2') = -18.000 -> this cell ENTERS the basis.
96 Closed loop: +(S2,D2) -> -(S2,D4) -> +(S3,D4) -> -(S3,D2)

```

Output (3/4): MODI Iteration 2 — optimal reached; final shipment plan

```

transportation_vam_modi > sample_output.txt
86 Current allocation:
87
88 S1 5 2
89 S2 7 2
90 S3 8 10
91 u_i : {'S1': 0, 'S2': 50, 'S3': 10}
92 v_j : {'D1': 19, 'D2': -2, 'D3': -10, 'D4': 10}
93 Opportunity costs d_ij = c_ij - (u_i+v_j) for non-basic cells:
94 | D1 D2 D3 D4
95 S1 basic 32 60 basic
96 S2 1 -18 basic basic
97 S3 11 basic 70 basic
98
99 Most negative opportunity cost at ('S2', 'D2') = -18.000 -> this cell ENTERS the basis.
100 Closed loop: +(S2,D2) -> -(S2,D4) -> +(S3,D4) -> -(S3,D2)
101 theta (max units shiftable) = min allocation among '-' cells = 2.000
102 Leaving cell (drops to 0): (S2, D4)
103 New allocation after reallocating 2.000 units around the loop:
104
105 --- MODI Iteration 2 ---
106 Current allocation:
107 | D1 D2 D3 D4
108 S1 5 2
109 S2 2 7
110 S3 6 12
111 u_i : {'S1': 0, 'S2': 32, 'S3': 10}
112 v_j : {'D1': 19, 'D2': -2, 'D3': 8, 'D4': 10}
113 Opportunity costs d_ij = c_ij - (u_i+v_j) for non-basic cells:
114 | D1 D2 D3 D4
115 S1 basic 32 42 basic
116 S2 19 basic basic 18
117 S3 11 basic 52 basic
118
119 All opportunity costs >= 0 -> current solution is OPTIMAL.
120
121 =====
122 FINAL RESULT - OPTIMAL SHIPMENT PLAN
123 =====
124 | D1 D2 D3 D4
125 S1 5 0 0 2
126 S2 0 2 7 0
127 S3 0 6 0 12
128
129 Shipments (source -> destination : units, cost):
130 S1 -> D1 : 5.000 units x 19 = 95.000
131 S1 -> D4 : 2.000 units x 10 = 20.000
132 S2 -> D2 : 2.000 units x 30 = 60.000
133 S2 -> D3 : 7.000 units x 40 = 280.000
134 S3 -> D2 : 6.000 units x 8 = 48.000
135 S3 -> D4 : 12.000 units x 20 = 240.000
136
137 MINIMUM TOTAL TRANSPORTATION COST = 743.000
138
139 =====
140

```

Output (4/4): minimum cost = 743, verified independently against SciPy

```
OT submission 13th Sept

transportation_vam_modi > sample_output.txt
111 u_i : { 'S1': 0, 'S2': 32, 'S3': 10 }
112 v_j : { 'D1': 19, 'D2': -2, 'D3': 8, 'D4': 10 }
113 Opportunity costs d_ij = c_ij - (u_i+v_j) for non-basic cells:
114 |   | D1  D2  D3  D4
115 S1 | basic 32  42  basic
116 S2 | 19  basic  basic  18
117 S3 | 11  basic  52  basic
118
119 All opportunity costs >= 0 -> current solution is OPTIMAL.
120
121 =====
122 FINAL RESULT - OPTIMAL SHIPMENT PLAN
123 =====
124 |   | D1  D2  D3  D4
125 S1 | 5   0   0   2
126 S2 | 0   2   7   0
127 S3 | 0   6   0  12
128
129 Shipments (source -> destination : units, cost):
130 S1 -> D1 : 5.000 units x 19 = 95.000
131 S1 -> D4 : 2.000 units x 10 = 20.000
132 S2 -> D2 : 2.000 units x 30 = 60.000
133 S2 -> D3 : 7.000 units x 40 = 280.000
134 S3 -> D2 : 6.000 units x 8 = 48.000
135 S3 -> D4 : 12.000 units x 20 = 240.000
136
137 MINIMUM TOTAL TRANSPORTATION COST = 743.000
138
139 =====
140 SciPy linprog verification
141 =====
142 Status : Optimization terminated successfully. (HiGHS Status 7: Optimal)
143 Optimal shipment plan (rows=S1..S3, cols=D1..D4):
144 [[ 5.  0.  0.  2.]
145  [ 0.  2.  7.  0.]
146  [ 0.  6.  0. 12.]]
147
148 Minimum total cost (SciPy) = 743.000
149 Expected from VAM + MODI implementation: 743.000
150
151
```